

DEMONSTRATING SQL PROFICIENCY

From Data Retrieval to Advanced Analytics in E-Commerce

Yash Jagdish Sonawane

Database Querying Portfolio

PROJECT CONTEXT & SCHEMA DESIGN

- **Objective:** Showcases structured problem-solving using relational database queries.
- **Core Schema:** Leverages a normalized three-table system simulating an e-commerce ecosystem.
- **Entities Tracked:** Manages critical master records (Customers) and transactional history (Sales) and Product specifics (Products)

Products	
COLUMNS	DATATYPE
Product_ID	text
Product_Name	text
Category	text
Brand	text
Original_Price	double
Discount_Percent	int
Discount_Amount	double
Selling_Price	double
Stock_Quantity	int
Weight_kg	double
Avg_Rating	double
Total_Reviews	int

Customer	
COLUMNS	DATATYPE
Customer_ID	text
Customer_Name	text
Gender	text
Age	int
Age_Group	text
Date_of_Birth	text
Email	text
Phone	text
City	text
State	text
Pincode	int
Registration_Date	text
Customer_Tier	text
Total_Orders	int
Total_Spent	double

Sales	
COLUMNS	DATATYPE
Order_ID	text
Customer_ID	text
Product_ID	text
Order_Date	text
Order_Time	text
Delivery_Date	text
Quantity	text
Unit_Price	text
Order_Value	text
Shipping_Cost	text
Coupon_Code	text
Coupon_Discount	text
Total_Amount	text
Payment_Mode	text
Order_Status	text
Rating	text
Review_Text	text
City	text
State	text
Customer_Age	text
Customer_Age_Group	text

DATA INGESTION – IMPORTING CSV RECORDS

- **Rapid Schema Generation:** Utilized the graphical **Table Data Import Wizard** on blank, data-less CSV headers to map out target database definitions instantly.
- **Security Error Resolution:** Diagnosed and bypassed security restrictions by enabling the required data stream variables across both server and client environments.
- **Server Activation:** Executed administrative overrides (SET GLOBAL local_infile = 1;) to permit file transmissions through the database subsystem.
- **Client Connection Handshake:** Initialized the query engine terminal with explicit client flag clearance (OPT_LOCAL_INFILE=1) to grant local drive read permissions.
- **High-Velocity Processing:** Executed high-performance engine loading (LOAD DATA LOCAL INFILE) to stream raw text records into tables in milliseconds.

The screenshot displays the MySQL Workbench interface. The SQL editor contains the following queries:

```
5 • TRUNCATE products;
6
7 • SET GLOBAL local_infile = 1;
8
9 • LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/products.csv'
10 INTO TABLE products
11 FIELDS TERMINATED BY ','
12 OPTIONALLY ENCLOSED BY ''
13 LINES TERMINATED BY '\n'
14 IGNORE 1 LINES;
15
16 • select*from products; -- TO AVOID HEADERS FROM CSV
```

The **Result Grid** shows the following data:

Product_ID	Product_Name	Category	Brand	Original_Price	Discount_Percent	Discount_Amount	Selling_Price	Stock_Quantity	Weight_kg	Avg_Rating	Total_Reviews
PROD000001	Samsung Mobile V5	Electronics	Samsung	130561.34	23	30029.11	100532.23	159	2.63	4.49	102
PROD000002	HP Laptop V5	Electronics	HP	36494.92	34	12408.27	24086.65	54	4.19	4.32	107
PROD000003	boAt Headphones V2	Electronics	boAt	106587.94	42	44766.93	61821.01	638	1.48	4.31	99
PROD000004	Noise Watch V7	Electronics	Noise	81275.27	47	38199.38	43075.89	572	3.3	4.42	127
PROD000005	Apple iPhone V4	Electronics	Apple	13099.36	43	5632.72	7466.64	865	3.71	4.37	98
PROD000006	Levis Jeans V1	Fashion	Levis	3330.44	9	299.74	3030.7	380	2.64	4.47	81
PROD000007	Peter England Shirt V8	Fashion	Peter	7504.54	12	900.54	6604	800	2.12	4.37	82
PROD000008	Nike Shoes V2	Fashion	Nike	7210.81	38	2740.11	4470.7	321	1.71	4.29	104
PROD000009	Titan Watch V10	Fashion	Titan	8545.82	48	4101.99	4443.83	266	0.82	4.52	94
PROD000010	Puma T-shirt V7	Fashion	Puma	12528.96	35	4385.14	8143.82	834	3.12	4.37	89

The **Output** pane shows the following actions and their results:

#	Time	Action	Message	Duration / Fetch
9	05:12:58	SELECT*FROM customers LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
10	05:13:06	SELECT*FROM products LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
11	05:13:09	TRUNCATE products	0 row(s) affected	0.078 sec
12	05:13:19	LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/products - Copy.csv' INTO TABLE products	3 row(s) affected Records: 3 Deleted: 0 Skipped: 0 Warnings: 0	0.015 sec
13	05:13:27	TRUNCATE products	0 row(s) affected	0.047 sec
14	05:13:34	LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/products.csv' INTO TABLE products	2000 row(s) affected Records: 2000 Deleted: 0 Skipped: 0 Warnings: 0	0.094 sec
15	05:13:42	select*from products LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

LEVEL 1: BASIC FILTERING & SORTING

- **Concepts Shown:** Implements fundamental data filtration using SELECT, FROM, WHERE, GROUP BY and ORDER BY.
- **Business Use:** Narrows down sprawling customer databases to look only at localized target markets.
- **Sorting Logic:** Alphabetizes output records strictly to improve readability for business end-users.

The screenshot displays a SQL IDE interface with a script editor at the top and a results/output pane at the bottom.

Script Editor:

```
28 • SELECT * FROM sales;
29 • Describe sales;
30
31 -- Changing name of Column i>Customer_ID to Customer_ID in customers table
32 • ALTER TABLE customers
33   RENAME COLUMN i>Customer_ID to Customer_ID;
34
35 -- Fetching all the customers located in Mumbai
36 • SELECT Customer_Name, State
37   FROM customers
38  WHERE State = 'Punjab'
39  GROUP BY Customer_Name
40  ORDER BY Customer_Name ASC;
41
42
```

Result Grid:

Customer_Name	State
Aadhya Agarwal	Punjab
Aadhya Gupta	Punjab
Aadhya Iyer	Punjab
Aadhya Joshi	Punjab
Aadhya Kulkarni	Punjab
Aadhya Kumar	Punjab
Aadhya Mehta	Punjab

Output:

#	Time	Action	Message	Duration / Fetch
✓ 16	04:26:33	SELECT * FROM customers LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
✓ 17	04:28:35	ALTER TABLE customers RENAME COLUMN i>Customer_ID to Customer_ID	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
✓ 18	04:28:41	SELECT Customer_ID, Customer_Name, State FROM customers WHERE State = 'Mumbai' ORDER BY Custo...	0 row(s) returned	0.110 sec / 0.000 sec
✓ 19	04:28:54	SELECT * FROM customers LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
✓ 20	04:29:15	SELECT Customer_ID, Customer_Name, State FROM customers WHERE State = 'Punjab' ORDER BY Custo...	1000 row(s) returned	0.047 sec / 0.000 sec
✗ 21	04:30:44	SELECT Customer_ID, Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Custo...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated colu...	0.000 sec
✓ 22	04:32:14	SELECT Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Customer_Name OR...	906 row(s) returned	0.032 sec / 0.000 sec

LEVEL 2: METRICS AGGREGATION & GROUP MATCHING

- **Concepts Shown:** Leverages GROUP BY to summarize counts and HAVING to isolate criteria post-aggregation.
- **Business Use:** Pinpoints geographic regions containing a high density of users to guide logistical planning.
- **Technical Rule:** Uses HAVING instead of WHERE because filters are applied to an aggregated column (COUNT).

The screenshot displays a SQL IDE interface with two queries and their results.

Query 1 (Lines 35-41): Fetching all the customers located in Mumbai. The query is: `SELECT Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Customer_Name ORDER BY Customer_Name ASC;`

Query 2 (Lines 42-49): Identifying high-volume market hubs with multiple customer profiles. The query is: `SELECT*FROM customers; SELECT State, COUNT(Customer_ID) AS Total_Customers FROM customers GROUP BY State HAVING COUNT(Customer_ID)>10 ORDER BY COUNT(Customer_ID) DESC;`

Result Grid: The first query result shows a table with columns State and Total_Customers. The data is as follows:

State	Total_Customers
UP	5218
Haryana	5124
Rajasthan	5057
Punjab	4083
Maharashtra	4050
Gujarat	4026
Tamil Nadu	3170

Action Output: The second query result shows a table with columns #, Time, Action, Message, and Duration / Fetch. The data is as follows:

#	Time	Action	Message	Duration / Fetch
19	04:28:54	SELECT*FROM customers LIMIT 5	5 row(s) returned	0.000 sec / 0.000 sec
20	04:29:15	SELECT Customer_ID, Customer_Name, State FROM customers WHERE State = 'Punjab' ORDER BY Custo...	1000 row(s) returned	0.047 sec / 0.000 sec
21	04:30:44	SELECT Customer_ID, Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Custo...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains nonaggregated colu...	0.000 sec
22	04:32:14	SELECT Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Customer_Name OR...	906 row(s) returned	0.032 sec / 0.000 sec
23	04:41:50	SELECT*FROM customers LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
24	04:45:13	SELECT State, COUNT(Customer_ID) AS Total_Customers FROM customers GROUP BY State HAVING COU...	Error Code: 1054. Unknown column 'Total_Custymers' in 'order clause'	0.000 sec
25	04:45:27	SELECT State, COUNT(Customer_ID) AS Total_Customers FROM customers GROUP BY State HAVING COU...	10 row(s) returned	0.094 sec / 0.000 sec

LEVEL 3: CROSS-TABLE RELATIONAL JOINS

- **Concepts Shown:** Executes an explicit INNER JOIN linking foreign keys across discrete database silos.
- **Business Use:** Bridges structural boundaries to calculate immediate revenue maps directly against consumer profiles.
- **Data Integrity:** Omits unfulfilled or cancelled items using hard conditional boundaries (Order_Status = 'Cancelled').

The screenshot displays the SQL Project IDE interface. The top pane shows a SQL query with line numbers 47 to 58. The query is as follows:

```
47 GROUP BY State
48 HAVING COUNT(Customer_ID)>10
49 ORDER BY COUNT(Customer_ID) DESC;
50
51 -- Merging transaction tables directly with customer identifiers
52 -- Identifying the Customers with Order status as "Cancelled"
53 • SELECT c.Customer_ID, c.Customer_Name, s.Order_Status, s.Total_Amount
54 from customers c
55 INNER JOIN sales s
56 ON c.Customer_ID = s.Customer_ID
57 WHERE s.Order_Status = 'Cancelled'
58 ORDER BY s.Total_Amount DESC;
```

The bottom pane shows the 'Result Grid' with 7 columns: Customer_ID, Customer_Name, Order_Status, and Total_Amount. It displays 8 rows of data for cancelled orders, sorted by Total_Amount in descending order.

Customer_ID	Customer_Name	Order_Status	Total_Amount
CUST00004643	Megha Sharma	Cancelled	9987.48
CUST00033748	Abhishek Pandey	Cancelled	9985.88
CUST00021103	Simran Verma	Cancelled	9985.88
CUST00009338	Imran Siddiqui	Cancelled	9985.88
CUST00006979	Aditya Reddy	Cancelled	997.61
CUST00027513	Pooja Mishra	Cancelled	9966.64
CUST00015143	Harsh Kumar	Cancelled	99605.56
CUST00072077	Karan Mishra	Cancelled	99605.56

Below the result grid, the 'Output' pane shows the 'Action Output' table, which logs the execution of various SQL commands and their results.

#	Time	Action	Message	Duration / Fetch
6	16:26:31	LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/products.csv' INTO TABLE product...	2000 row(s) affected Records: 2000 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec
7	16:26:31	SELECT*FROM products LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
8	16:26:31	SELECT*FROM sales LIMIT 0, 1000	1000 row(s) returned	0.016 sec / 0.000 sec
9	16:26:31	TRUNCATE sales	0 row(s) affected	0.063 sec
10	16:26:31	LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/sales.csv' INTO TABLE sales FIEL...	250000 row(s) affected Records: 250000 Deleted: 0 Skipped: 0 Warnings: 0	5.734 sec
11	16:26:36	SELECT*FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
12	16:26:36	Describe sales	21 row(s) returned	0.000 sec / 0.000 sec
13	16:26:36	ALTER TABLE customers RENAME COLUMN 'Customer_ID' to Customer_ID	Error Code: 1054. Unknown column 'Customer_ID' in 'customers'	0.000 sec
14	16:29:33	SELECT c.Customer_ID, c.Customer_Name, s.Order_Status, s.Total_Amount from customers c INNER JOIN s...	1000 row(s) returned	0.563 sec / 0.000 sec

LEVEL 4 – ADVANCED ANALYTICS (WINDOW FUNCTIONS)

- **Concepts Shown:** Applies `DENSE_RANK()` OVER (PARTITION BY... analytical computation mechanics.
- **Business Use:** Tracks and ranks user purchasing frequencies to isolate top VIP spenders from casual consumers.
- **Granular Output:** Computes contextual positions safely across sections without losing structural integrity or collapsing granular line rows.

The screenshot displays a SQL IDE interface with a query editor and a results pane. The query editor shows a SQL query that joins a 'customers' table with a 'sales' table, filters for 'Cancelled' orders, and uses a window function to rank orders by value within each customer partition.

```
51 -- Merging transaction tables directly with customer identifiers
52 -- Identifying the Customers with Order status as "Cancelled"
53 • SELECT c.Customer_ID, c.Customer_Name, s.Order_Status, s.Total_Amount
54 from customers c
55 INNER JOIN sales s
56 ON c.Customer_ID = s.Customer_ID
57 HAVING s.Order_Status = 'Cancelled'
58 ORDER BY s.Total_Amount DESC;
59
60 -- Isolating high-value transactions relative to individual accounts
61 • SELECT Customer_ID, Order_ID, Order_Value,
62 DENSE_RANK() OVER (PARTITION BY Customer_ID ORDER BY Order_Value DESC) as Order_Rank
63 FROM sales;
```

The results pane shows a table with the following data:

Customer_ID	Order_ID	Order_Value	Order_Rank
CUST00000001	ORD0000147021	34017.19	1
CUST00000001	ORD0000169535	2721.44	2
CUST00000001	ORD0000231581	1808.22	3
CUST00000001	ORD0000170293	166.74	4
CUST00000001	ORD0000118893	1298.13	5
CUST00000002	ORD0000088793	838.15	1
CUST00000002	ORD0000018546	745.1	2
CUST00000002	ORD0000002149	4238.68	3
CUST00000002	ORD0000121764	3549.53	4

The bottom pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
10	04:00:23	LOAD DATA LOCAL INFILE 'C:/Users/Yash/Downloads/SQL Project file/sales.csv' INTO TABLE sales FIEL...	250000 row(s) affected Records: 250000 Deleted: 0 Skipped: 0 Warnings: 0	11.172 sec
11	04:00:34	SELECT * FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
12	04:00:34	Describe sales	21 row(s) returned	0.000 sec / 0.000 sec
13	04:00:34	ALTER TABLE customers RENAME COLUMN i » Customer_ID to Customer_ID	Error Code: 1054. Unknown column 'i » Customer_ID' in 'customers'	0.000 sec
14	04:00:54	SELECT c.Customer_ID, c.Customer_Name, s.Order_Status, s.Total_Amount from customers c INNER JOIN s...	1000 row(s) returned	2.234 sec / 0.563 sec
15	04:06:22	SELECT * FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
16	04:09:56	SELECT Customer_ID, Order_ID, Order_Value, DENSE_RANK() OVER (PARTITION BY Customer_ID ORDE...	250000 row(s) returned	0.907 sec / 0.641 sec

LEVEL 5: CROSS-FUNCTIONAL REVENUE & VALUE SEGMENTATION

- **Concepts Shown:** Executes multi-table INNER JOIN bridges across three distinct data environments while applying numeric explicit conversions (CAST).
- **Business Use:** Identifies high-value intersection points between specific customer loyalty tiers and product categories to optimize high-margin inventory distribution.
- **Data Integrity:** Isolates true realized sales by screening out unfulfilled pipelines and applying strict validation rules (Order_Status = 'Delivered').

The screenshot displays a SQL IDE interface with a query editor and a results pane. The query editor shows a SQL query that joins sales, customers, and products tables, filters for 'Delivered' orders, and calculates revenue by customer tier and category. The results pane shows a table with columns Customer_Tier, Category, and REVENUE. The results are as follows:

Customer_Tier	Category	REVENUE
Platinum	Electronics	339105506.54
Platinum	Home	463872328.79
Platinum	Sports	344919364.17
Platinum	Fashion	214743406.27
Gold	Home	96703692.67

The results pane also shows a log of database actions and their execution times. The log includes the following entries:

#	Time	Action	Message	Duration / Fetch
26	17:26:40	SELECT Customer_Name, State FROM customers WHERE State = 'Punjab' GROUP BY Customer_Name OR...	906 row(s) returned	0.125 sec / 0.000 sec
27	17:26:40	SELECT*FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
28	17:26:40	SELECT State, COUNT(Customer_ID) AS Total_Customers FROM customers GROUP BY State HAVING COU...	10 row(s) returned	0.109 sec / 0.000 sec
29	17:26:40	SELECT c.Customer_ID, c.Customer_Name, s.Order_Status, s.Total_Amount from customers c INNER JOIN s...	1000 row(s) returned	0.484 sec / 0.000 sec
30	17:26:40	SELECT Customer_ID, Order_ID, Order_Value, DENSE_RANK() OVER (PARTITION BY Customer_ID ORDE...	250000 row(s) returned	0.953 sec / 0.625 sec
31	17:26:43	SELECT Customer_ID, Order_ID, Order_Value FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
32	17:26:43	SELECT*FROM sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
33	17:26:43	SELECT c.Customer_Tier, p.Category, SUM(CAST(s.Total_Amount AS DECIMAL(10,2))) AS REVENUE FROM...	Error Code: 2013. Lost connection to MySQL server during query	30.000 sec
34	17:28:05	SELECT c.Customer_Tier, p.Category, SUM(CAST(s.Total_Amount AS DECIMAL(10,2))) AS REVENUE FROM...	21 row(s) returned	2.578 sec / 0.000 sec

LEVEL 6: COHORT IDENTIFICATION & MARKETING RE-ENGAGEMENT

- **Concepts Shown:** Implements an exclusive relational LEFT JOIN combined with an identifying null-space filter (IS NULL).
- **Business Use:** Pinpoints exact registration cohorts that exhibit high friction or zero adoption rates to serve as targets for targeted promo re-engagement.
- **Optimization Note:** Avoids heavy operational subqueries by leveraging indexed left boundary filters for maximum query performance.

The screenshot displays a SQL Project IDE interface. The top pane shows a SQL query with two main parts: a query for delivered orders and a query for isolating registered user accounts with zero transactional history. The bottom pane shows the results of the second query, which is a table with columns: Customer_ID, Customer_ID, Phone, State, and Registration_Date. The results show 6 rows of data for customers in Haryana, UP, Gujarat, Rajasthan, and Punjab.

```
SQL Project x
Limit to 1000 rows
70 INNER JOIN customers c ON s.Customer_ID = c.Customer_ID
71 INNER JOIN products p ON s.Product_ID = p.Product_ID
72 WHERE s.Order_Status = 'Delivered'
73 GROUP BY c.Customer_Tier, p.Category
74 ORDER BY Revenue Desc;
75
76 -- Isolating registered user accounts with zero transactional history
77 • SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date
78 FROM customers c
79 LEFT JOIN sales s
80 ON c.Customer_ID = s.Customer_ID
81 WHERE s.Order_ID IS NULL
82 ORDER BY c.Registration_Date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Customer_ID	Customer_ID	Phone	State	Registration_Date
▶	CUST00012247	CUST00012247	+91-7811516724	Haryana	2023-06-14
	CUST00037089	CUST00037089	+91-9794673746	UP	2023-06-15
	CUST00009542	CUST00009542	+91-9791985778	Gujarat	2023-06-16
	CUST00036739	CUST00036739	+91-9505032702	Rajasthan	2023-06-17
	CUST00022124	CUST00022124	+91-7540518638	Punjab	2023-06-20

Result 24 x | Read On

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 34	17:28:05	SELECT c.Customer_Tier, p.Category, SUM(CAST(s.Total_Amount AS DECIMAL(10,2))) AS REVENUE FROM...	21 row(s) returned	2.578 sec / 0.000 sec
✓ 35	17:38:26	SELECT * FROM customers LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
✗ 36	17:42:18	SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date FROM customer c LEFT JOIN...	Error Code: 1146. Table 'dummy1.customer' doesn't exist	0.016 sec
✗ 37	17:42:29	SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date FROM customers c LEFT JOI...	Error Code: 1054. Unknown column 's.Order_ID' in 'having clause'	0.000 sec
✓ 38	17:42:49	SELECT * FROM Sales LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
✗ 39	17:43:46	SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date FROM customers c LEFT JOI...	Error Code: 1054. Unknown column 'c.Registertion_Date' in 'order clause'	0.015 sec
✗ 40	17:44:01	SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date FROM customers c LEFT JOI...	Error Code: 1054. Unknown column 'c.Registertion_Date' in 'order clause'	0.000 sec
✓ 41	17:44:14	SELECT * FROM customers LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
✓ 42	17:44:35	SELECT c.Customer_ID, c.Customer_ID, c.Phone, c.State, c.Registration_Date FROM customers c LEFT JOI...	86 row(s) returned	1.203 sec / 0.000 sec

SUMMARY OF DATABASE CAPABILITIES

End-to-End Enterprise Execution

- **Lifecycle Management:** Orchestrated complete data operations within an enterprise ecosystem, spanning normalized multi-table database architecture, secure raw CSV data ingestion, and advanced analytical calculations.
- **Production-Ready Standards:** Authored highly legible, scalable SQL scripts optimized with standardized keyword casing, explicit join relationships, and precise conditional indexing (WHERE).

Advanced Technical Problem-Solving

- **Engine Environment Controls:** Successfully diagnosed and bypassed restrictive directory transport variables by executing synchronized engine handshakes (local_infile = 1 and OPT_LOCAL_INFILE=1) across client-server lines.
- **Dynamic Type Transformations:** Safeguarded operational data integrity by applying runtime format overrides (CAST), migrating flat text attributes into mathematically viable decimal arrays (DECIMAL(10,2)) for exact processing.

Business Intelligence & Cohort Reporting

- **Granular Micro-Targeting:** Developed non-destructive window configurations (DENSE_RANK() OVER (PARTITION BY...)) to isolate, rank, and track repeat VIP buyer habits directly alongside micro-transaction records.
- **Relational Synthesis Reporting:** Leveraged multi-layered relational join frameworks across three discrete transaction pillars (Customers, Sales, and Products) to highlight revenue concentrations within distinct user tiers.
- **Leakage and Churn Identification:** Built high-value cohort analysis scripts using exclusive boundary conditions (LEFT JOIN ... WHERE ... IS NULL) to expose underperforming marketing profiles and inactive user groups

THANK YOU!

